```
%sql
--Note that we need to enable Change Data Feed on the table to create the index
CREATE TABLE IF NOT EXISTS evaluation_table (
  id_question BIGINT GENERATED BY DEFAULT AS IDENTITY,
  question_asked STRING,
  answer_given STRING,
  pdf_name STRING,
  article_name STRING
  ) TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

Figure 7.44 – Creating evaluation_table to store the question/answer pairs

We save the table and are now ready to evaluate our model.

```
df_eval.write.mode("append").saveAsTable(f"{catalog}.{database_name}.evaluation_table")
```

Figure 7.45 – Saving questions_evaluation.csv data to a Delta table

Setting up the evaluation workflow

We're now ready to open the second notebook, `CH7-02-Evaluating ChatBot`, and evaluate our chatbot against the ground truth labels we saved to `evaluation_table`. Let's briefly discuss how we will compare our model's outputs to the human-generated answers. While there is plenty of ongoing research in the realm of automated LLM evaluation methods, we will focus on one technique in particular: *LLM-as-a-judge*. This method brings in a second large language model to evaluate the performance of the first, automating the tedious task of comparing the generated answer to the true answer that a human would traditionally have to do.

To use an LLM as a judge, we first load the original model we created in *Chapter 6*.

```
import mlflow
from mlia_utils.mlflow_funcs import *
from pyspark.sql.functions import col, udf, length, pandas_udf, explode

os.environ['DATABRICKS_TOKEN'] = dbutils.secrets.get("mlaction", "rag_sp_token")
model_name = f"{catalog}.{database_name}.mlaction_chatbot_model"

model_version_to_evaluate = get_latest_model_version(model_name)
mlflow.set_registry_uri("databricks-uc")
rag_model = mlflow.langchain.load_model(f"models:/{model_name}/{model_version_to_evaluate}")
```

Figure 7.46 – Loading the mlaction_chatbot_model as rag_model